

0:00 – 0:45 | Introduction & Architecture (Slides 1–2)

[Slide 1] Good day, everyone. This is a presentation of a full-stack data pipeline designed to aggregate unstructured data from heterogeneous web resources into a single, unified search index [cite: 1].

The primary objective of this project is to demonstrate the underlying engineering required to extract, normalize, and index data from structurally different and occasionally hostile target environments [cite: 1].

[Slide 2] The application architecture implements a classic ETL pipeline. The backend is built using Node.js, Express, and TypeScript [cite: 1].

- **For Extraction:** we crawl data using Axios and Puppeteer [cite: 1].
- **For Transformation:** We normalize diverse structures into a single internal model called **RawVacancy** [cite: 1].
- **And for Loading:** Data is cached in Redis [cite: 1] and upserted into Elasticsearch [cite: 1] for real faceted search.
We also use PostgreSQL to store metadata, scraping logs, and process states [cite: 1].

0:45 – 1:45 | 4 Sources, 4 Problems (Slides 3–4)

[Slide 3] The core challenge of this project was to test a specific hypothesis: can we crawl completely different websites using a limited, unified set of tools? The answer is yes, thanks to the **Strategy pattern** [cite: 1].

[Slide 4] I selected 4 real-world sources, each presenting a unique challenge [cite: 1]:

1. **Habr Career:** The easiest case. It's fully server-rendered, and we extract data cleanly via Axios from a stable JSON-LD block.
2. **RemoteOK:** Gated behind Cloudflare. Standard HTTP requests get a 403 error. To bypass this, I use Puppeteer with a real desktop user-agent and browser fingerprint.
3. **WeWorkRemotely:** Headless Puppeteer got fingerprinted and blocked after a single detail-page navigation. My pivot was to switch to their official RSS feed, which isn't gated at all and is pulled via Axios.
4. **Craigslis:** No bot wall, but it's a general classifieds site, not a tek jobboard. To align it with other sources, I narrowed the scope down to the IT category across 4 specific cities.

1:45 – 2:15 | Defining Mapping (Slide 5)

[Slide 5] It's crucial to distinguish between two different types of mapping occurring in this pipeline:

- **ETL Mapping:** This happens at parse time inside each `CrawlStrategy` (Stage 2). It translates source-specific vocabularies, like `hiringOrganization.name`, into our uniform `company` field.
- **Elasticsearch Mapping:** This defines the index schema itself (Stage 4)—field types, keywords, and text sub-fields for autocomplete. This mapping is versioned. If we update the schema, the application automatically drops and rebuilds the index from scratch to prevent data corruption.

2:15 – 3:30 | Live Demo / Video Playback (Slide 6 or Screen Share)

[Slide 6] Let's move on to the live demonstration.

(Start your Zoom screen share or play your pre-recorded video here: 30 seconds of terminal logs, followed by 45 seconds of the UI)

Take a look at the terminal. The crawler is built to be "polite by design". Every source is rate-limited with jitter to avoid overloading the target servers [cite: 1].

Now, look at the search interface. This is the output of our Elasticsearch document. Data from four structurally different worlds is seamlessly unified. As I type, you can see instant autocomplete, match highlighting, and dynamic faceted filtering for locations and remote status.

3:30 – 4:00 | Conclusion (Slide 7)

[Slide 7] To wrap up, the biggest takeaway is that this architecture is highly scalable and maintainable. If we need to add a 5th or a 10th job board tomorrow, we don't need to change the core pipeline. An engineer simply writes a new isolated Strategy module that implements the `CrawlStrategy`

interface and maps the new fields into `RawVacancy` [cite: 1].

That concludes my presentation. Thank you for your time, and I'd be happy to take any questions!



Полезные фразы для ответов на вопросы (Q&A):

Если вас спросят про технические детали на английском, вот пара заготовок:

- «*Why Elasticsearch and not just PostgreSQL for search?*»

Ответ: "Elasticsearch was chosen specifically for its out-of-the-box support for faceted search, fuzzy matching, and fast full-text indexing, which is hard to scale in relational databases."

- «*How do you handle schema changes?*»

Ответ: "Since the ES index acts as a derived cache, we use index versioning. Any breaking schema change triggers an automatic index rebuild rather than an in-place migration."

Как вам такой вариант текста? Если нужно **упростить** какие-то предложения для более легкого произношения или, наоборот, сделать их **более сложными**, дайте знать!